# Sensors & Systems

Kalman Filter for Linear Systems

Torben Knudsen | Electronic Systems, 8th Semester

## The Three Problem Types: DD, CD and CC

Every Kalman filter model consists of exactly two equations that answer two completely different questions:

- **State equation** — how does the system evolve over time? This is purely physics: the state changes on its own, driven by any input you apply and disturbed by random process noise. Nothing is observed yet — it is just the world doing its thing.
- **Measurement equation** — what does the sensor read at a given moment? The sensor observes the current state, possibly with a known input, and adds its own measurement noise. That is all.

The choice of *continuous* or *discrete* for the state equation is a **modelling decision** based on how you want to describe the physics. If the physics are naturally described by differential equations (chemistry, mechanics) use a continuous state; if you are already thinking in time steps use a discrete state. The measurement is almost always discrete in practice because real sensors sample — so the measurement equation stays discrete regardless of what you chose for the state.

Before writing the Kalman filter equations it is important to know *which version* of the problem you are dealing with. The type is determined by whether the **state model** and the **measurements** are in discrete or continuous time.

| Type | State model | Measurements | Typical use |
|---|---|---|---|
| **DD** | Discrete time | Discrete time | Digital systems, embedded processors, sampled data. **This is the focus of this course.** |
| **CD** | Continuous time (SDE) | Discrete time | Physics modelled in continuous time but sensor sampled at fixed intervals. |
| **CC** | Continuous time | Continuous time | Theoretical; solution is the Kalman–Bucy filter. *Not needed in this course.* |

## DD — Discrete-Discrete

Both the physics model and the sensor are discrete — everything happens at fixed sampled steps.

$$x_{k+1} = \Phi_k x_k + \Gamma_k u_k + w_k, \quad w_k \sim \mathcal{N}(0, Q_k)$$
$$y_k = H_k x_k + D_k u_k + v_k, \quad v_k \sim \mathcal{N}(0, R_k)$$

| Symbol | Meaning |
|---|---|
| $x_k$ | State vector at step $k$ — the quantities you want to estimate (e.g. position, velocity, temperature). |
| $\Phi_k$ | State transition matrix — describes how the state naturally evolves from step $k$ to step $k{+}1$ with no input. |
| $\Gamma_k$ | Input matrix — maps the known control input $u_k$ into the state update. |
| $u_k$ | Known control input at step $k$ (e.g. heater power, motor voltage). |
| $w_k$ | Process noise — random disturbances acting on the state (wind, friction, unknown forces). $\sim \mathcal{N}(0, Q_k)$. |
| $Q_k$ | Process noise covariance — how large and correlated the disturbances are. |
| $y_k$ | Measurement (sensor reading) at step $k$. |
| $H_k$ | Measurement matrix — maps the state to what the sensor actually sees (e.g. $H = [1\ 0]$ means only position is measured, not velocity). |
| $D_k$ | Direct feedthrough matrix — the part of the input that appears directly in the measurement (often zero). |
| $v_k$ | Measurement noise — sensor inaccuracy. $\sim \mathcal{N}(0, R_k)$. |
| $R_k$ | Measurement noise covariance — how noisy the sensor is. |

The noise sequences $w_k$ and $v_k$ are **white** (independent in time) and **uncorrelated with each other**: $E(w_k v_l^\top) = 0$.

**Is this a requirement — what happens if it is violated?**
Yes — both conditions are *necessary* for the standard Kalman filter to be optimal (MMSE).

**If the noise is not white** (i.e. it is *coloured* — correlated across time steps, such as a slow drift): the filter is no longer optimal because it ignores the memory in the noise. The fix is to **augment the state** with a model of the coloured noise (e.g. model it as a first-order Markov process driven by white noise), making the combined noise effectively white again.

**If process and measurement noise are correlated** ($E(w_k v_k^\top) \neq 0$, e.g. both affected by the same physical source): the standard equations are wrong. A modified version of the KF exists that adds a cross-covariance term, but this is uncommon in

practice.

**Example — room temperature control.**
The temperature $T_k$ in a room is sampled every second by a digital thermometer. The heater model is discretised: $T_{k+1} = T_k + \alpha(T_{\text{set}} - T_k) \cdot T_s + w_k$, where $w_k$ is random disturbance (window opened, people entering). The thermometer reads $y_k = T_k + v_k$. Both the state update and the measurement happen at the same discrete 1 s steps — this is DD. The Kalman filter simply multiplies by $\Phi$ to predict and corrects with the thermometer reading.

### CD — Continuous-Discrete

The physics happen continuously (described by ODEs/SDEs), but the sensor only delivers measurements at discrete sample intervals.

**ODE vs SDE — what is the difference?**
An **ODE** (Ordinary Differential Equation) is a deterministic equation describing how a quantity changes over time: $\dot{x}(t) = f(x(t), t)$. Given exact initial conditions it always produces the same trajectory — no randomness. Example: Newton's second law $m\ddot{x} = F$ is an ODE.

An **SDE** (Stochastic Differential Equation) is an ODE with an added *random noise term*: $dx(t) = f(x,t)\,dt + g(x,t)\,dW(t)$, where $dW(t)$ is a Wiener process (Brownian motion) increment — continuous-time white noise. The same initial conditions produce a *different random trajectory* every time. SDEs are used when the physics are continuous but genuinely uncertain (turbulence, molecular diffusion, financial prices).

In the CD Kalman filter the state model is an SDE because real continuous systems always have some random disturbance acting on them.

$$dx(t) = \big(F(t)x(t) + B(t)u(t)\big)dt + d\omega(t), \quad \omega(t) \sim \mathcal{W}(Q(t))$$
$$y(t_k) = H_k x(t_k) + D_k u(t_k) + v_k, \quad v_k \sim \mathcal{N}(0, R_k)$$

| Symbol | Meaning |
|---|---|
| $x(t)$ | State as a *continuous* function of time. |
| $F(t)$ | Continuous-time system matrix — equivalent of $\Phi$ but for a continuous ODE (not a discrete step). |
| $B(t)$ | Continuous-time input matrix — equivalent of $\Gamma$. |
| $u(t)$ | Continuous control input. |
| $d\omega(t)$ | Continuous-time process noise increment (Wiener process / Brownian motion) — the continuous equivalent of $w_k$. |
| $Q(t)$ | Spectral density of the continuous noise (not the same units as discrete $Q_k$). |
| $y(t_k)$ | Discrete measurement arriving at sample time $t_k$ — same structure as DD. |
| $H_k$, $D_k$, $v_k$, $R_k$ | Same meaning as in DD, applied at the discrete sample instant $t_k$. |

**Example — chemical reactor.**

- **Input** $u(t)$: the flow rate of reactant you pump into the reactor. It is continuous — you are continuously pumping.
- **State** $x(t)$: the concentration inside the reactor. It evolves continuously because chemistry does not pause between readings: $\dot{c}(t) = -k\,c(t) + u(t) + w(t)$.

- **Measurement / output** $y(t_k)$: a lab technician scoops out a sample *every 30 minutes* and measures the concentration. That scoop only happens at discrete moments — so the measurement is discrete even though the concentration itself never stopped changing: $y_k = c(t_k) + v_k$.

The state is continuous, the measurement is discrete — this is CD.

**Key difference from DD: you must integrate the ODE between measurements.**
In DD the prediction step is just $\hat{x}_{k+1|k} = \Phi_k \hat{x}_{k|k}$ — one matrix multiply.
In CD, between two measurement times $t_{k-1}$ and $t_k$ the state has been evolving continuously according to the ODE. You cannot just multiply by $\Phi$; you must *integrate the continuous differential equation forward* from $t_{k-1}$ to $t_k$ to get the predicted state and covariance. For a linear time-invariant system this integral has a closed-form solution (matrix exponential); in general it requires a numerical ODE solver. The measurement update step is then the same as in DD.

### CC — Continuous-Continuous

Both the state model and the measurements are continuous — no sampling anywhere.

In practice CC is almost never implemented directly on hardware because all modern processors sample. It exists as the **theoretical foundation** from which DD and CD are derived — understanding CC gives you the intuition, but for any real digital system you will use DD or CD.

**Which type should I use in practice?**
Almost all real implementations run on a digital processor and sample sensors at fixed intervals. That makes the **DD Kalman filter** the right choice. If the physics are best described in continuous time (e.g. a mechanical system), you discretise the continuous model first (using zero-order hold or Forward Euler) and then apply the DD filter. The CD filter exists for cases where very accurate discretisation of a stiff SDE is needed between measurement steps.

# The DD Kalman Filter Algorithm

This is the complete algorithm — everything you need to implement a Kalman filter for a discrete-time linear system.

## Notation

The subscript notation keeps track of *which data was used* when computing an estimate:

| Symbol | Meaning | Plain words |
|---|---|---|
| $\hat{x}_{k\|k}$ | $E(x_k \mid Y_k)$ | Best estimate of $x$ at time $k$, using all data up to and including step $k$. |
| $\hat{x}_{k\|k-1}$ | $E(x_k \mid Y_{k-1})$ | Predicted estimate of $x$ at time $k$, using data only up to step $k-1$ (before new measurement arrives). |
| $P_{k\|k}$ | $E(\tilde{x}_{k\|k}\,\tilde{x}_{k\|k}^\top)$ | Error covariance *after* measurement update. |
| $P_{k\|k-1}$ | $E(\tilde{x}_{k\|k-1}\,\tilde{x}_{k\|k-1}^\top)$ | Error covariance *before* measurement (prediction uncertainty). |
| $\tilde{y}_{k\|k-1}$ | $y_k - \hat{y}_{k\|k-1}$ | Innovation: how surprising is the new measurement? |

## Initialisation

$$x_0 \sim \mathcal{N}(\hat{x}_{0|-1},\ P_{0|-1})$$

Provide a starting guess $\hat{x}_{0|-1}$ for the state and $P_{0|-1}$ for the uncertainty. If you have no prior knowledge set $\hat{x}_{0|-1} = 0$ and $P_{0|-1} = I$.

## Step A — Measurement update (run when $y_k$ arrives)

A new measurement has just arrived. Correct the predicted estimate using the new information:

$$\hat{y}_{k|k-1} = H_k \hat{x}_{k|k-1} + D_k u_k \qquad \text{(predicted measurement)}$$

$$\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1} \qquad \text{(innovation)}$$

$$K_k = P_{k|k-1} H_k^\top \left( H_k P_{k|k-1} H_k^\top + R_k \right)^{-1} \qquad \text{(Kalman gain)}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_{k|k-1} \qquad \text{(updated state estimate)}$$

$$P_{k|k} = \left( I - K_k H_k \right) P_{k|k-1} \left( I - K_k H_k \right)^\top + K_k R_k K_k^\top \qquad \text{(updated covariance)}$$

### Step B — Time update (prediction to next step)

Propagate the current estimate forward to step $k{+}1$ using the dynamics model:

$$\hat{x}_{k+1|k} = \Phi_k \hat{x}_{k|k} + \Gamma_k u_k \qquad \text{(predicted state)}$$

$$P_{k+1|k} = \Phi_k P_{k|k} \Phi_k^\top + Q_k \qquad \text{(predicted covariance)}$$

Then wait for $y_{k+1}$ and repeat Step A.

**What each equation is doing in plain language.**

- **Innovation** $\tilde{y}$: the gap between what the sensor actually read and what the filter predicted it would read. A large gap means the prediction was wrong — trust the measurement.
- **Kalman gain** $K_k$: balances measurement noise ($R_k$) against prediction uncertainty ($P_{k|k-1}$). Large $R_k$ (noisy sensor) $\Rightarrow$ small $K_k$ (ignore measurement, trust prediction). Large $P_{k|k-1}$ (uncertain prediction) $\Rightarrow$ large $K_k$ (trust measurement).
- **Updated state**: old prediction nudged towards the measurement by fraction $K_k$.
- **Updated covariance**: shrinks because we just learned something. The $(I - K_k H_k)$ term reduces the uncertainty that was already in $P_{k|k-1}$.
- **Predicted covariance**: grows by $Q_k$ because the unknown process noise $w_k$ adds fresh uncertainty every step.

## Block Diagram Interpretation

The Kalman filter runs two parallel loops — one for the **mean value** (the state estimates) and one for the **covariance** (the uncertainties). They are shown separately below.

### Mean value part

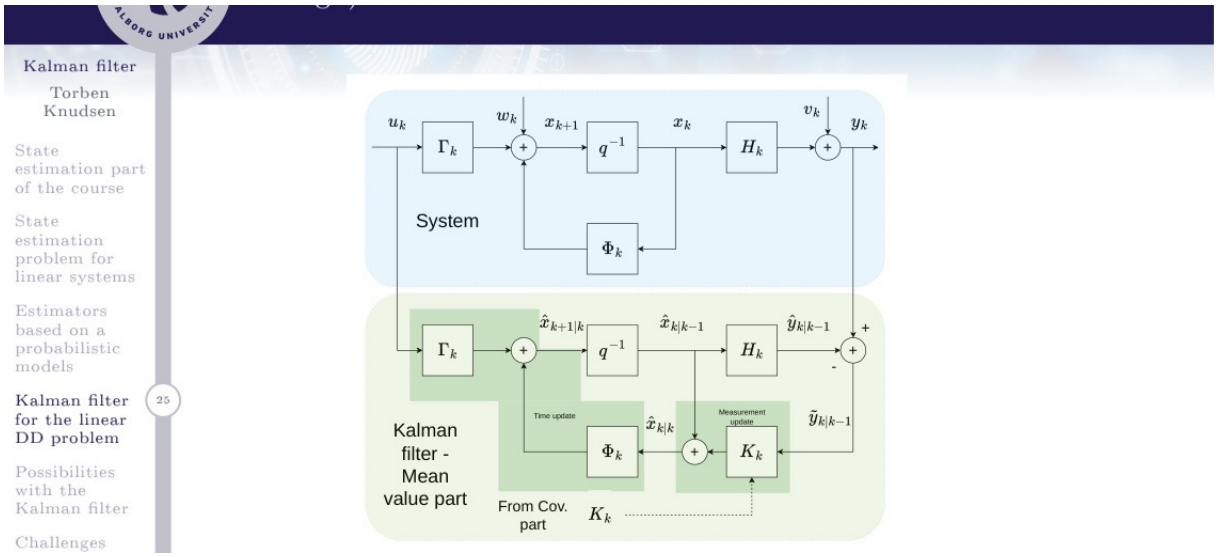


Figure 1: Block diagram of the real system (blue, top) and the Kalman filter mean-value part (green, bottom). The filter is a copy of the system dynamics driven by the same input $u_k$, corrected at each step by the Kalman gain $K_k$ applied to the innovation $\tilde{y}_{k|k-1}$.

The key insight from the diagram: the Kalman filter is essentially a **model of the system running in parallel**. It predicts where the state should be ($\hat{x}_{k|k-1}$ via $\Phi_k$ and $\Gamma_k$), compares the prediction to the actual measurement ($\tilde{y}_{k|k-1}$), and adds a correction proportional to that surprise ($K_k\tilde{y}_{k|k-1}$). If the model were perfect and there were no noise, the correction would always be zero.

> **The filter is a copy of the system.**
> Notice the bottom (green) box contains exactly the same blocks as the top (blue) system: $\Phi_k$ (state transition), $\Gamma_k$ (input matrix), $H_k$ (output), and a $q^{-1}$ delay. The only additions are $K_k$ (the correction gain) and the subtraction node that forms the innovation $\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$. The gain $K_k$ is fed in from the covariance part (dashed arrow).

## Covariance part

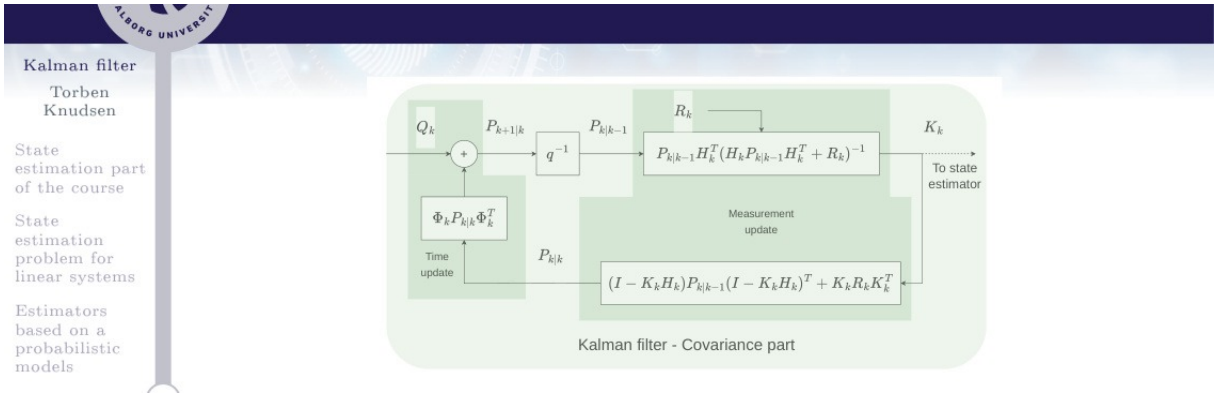


Figure 2: Block diagram of the Kalman filter covariance part. It runs independently of the measurements and feeds the gain $K_k$ to the mean-value part.

The covariance loop runs **independently of the measurements** — it only uses the model matrices ($\Phi_k$, $H_k$, $Q_k$, $R_k$), not the actual sensor data $y_k$. It propagates $P_{k|k-1}$ forward using the time update, then shrinks it via the measurement update, and outputs the Kalman gain $K_k$ to the state estimator.

**Consequence: the gain can be pre-computed.**
Because $K_k$ and $P_{k|k}$ do not depend on measurements — only on the system model and noise matrices — for a **time-invariant** system ($\Phi$, $H$, $Q$, $R$ all constant) the covariance converges to a steady state $P_\infty$ and the gain converges to a constant $K_\infty$. You can solve for these *offline* before the filter even runs, using the **Discrete Algebraic Riccati Equation (DARE)**:

$$P = \Phi P \Phi^\top - \Phi P H^\top (H P H^\top + R)^{-1} H P \Phi^\top + Q$$

At steady state the filter becomes a fixed-gain observer — much cheaper to run in real time.

## Key Properties of the Kalman Filter

---

### 1 — Optimal in the MMSE sense

For a **linear system with Gaussian noise**, the Kalman filter is the **minimum mean-square error (MMSE)** estimator. No other estimator can produce a lower expected squared error. This is not a heuristic — it follows directly from the fact that the optimal estimate of $x$ given $y$ is the conditional mean $E(x \mid y)$, and for jointly Gaussian distributions this conditional mean is exactly what the Kalman filter computes.

### 2 — The estimate is unbiased

$$E(x_k - \hat{x}_{k|k}) = 0$$

On average the filter makes no systematic error. A biased estimator would consistently over- or under-estimate, which would be easy to exploit — so unbiasedness is a basic requirement for any good estimator.

### 3 — Orthogonality principle

The estimation error is **uncorrelated with the measurements**:

$$E\big[(x_k - \hat{x}_{k|k})(y_j - E(y_j))^\top\big] = 0 \quad \forall j \le k$$

In plain words: there is no remaining information in the measurements that the filter has not already used. If the error *were* correlated with $y$, you could improve the estimate by adjusting for it — so zero correlation is the hallmark of an optimal estimator.

### 4 — Innovation sequence is white

The innovations $\tilde{y}_{k|k-1}$ at different time steps are **mutually uncorrelated**:

$$E\big[\tilde{y}_{k|k-1}\,\tilde{y}_{j|j-1}^\top\big] = 0 \quad \text{for } k \neq j$$

> **Why this matters for fault detection.**
> If the filter model is correct, the innovations should look like white noise. If the innovations are *correlated* across time steps — e.g. consistently positive for several steps in a row — it means either the model is wrong, the noise matrices $Q$ or $R$ are misspecified, or the sensor has drifted. Monitoring the innovation sequence (called **whiteness testing**) is one of the main tools for detecting sensor faults and model mismatch in real systems.

### 5 — Gain and covariance are measurement-independent

$K_k$, $P_{k|k}$, $P_{k|k-1}$ depend only on the model matrices $\Phi_k$, $H_k$, $Q_k$, $R_k$ — not on the actual measurement values $y_k$. This means the covariance loop (and the gain schedule) can be computed offline and stored, making real-time implementation very efficient.

## Possibilities and Challenges

### What the Kalman filter can be used for

The Kalman filter is a flexible tool — the creative part is choosing what goes into the state vector $x_k$:

| Application | How it works |
|---|---|
| State estimation | Classic use: estimate position, velocity, orientation from noisy sensor data. |
| Signal filtering | Put the signal model in the state; use KF to smooth noisy measurements (equivalent to an optimal Wiener filter). |
| Parameter estimation | Augment the state with unknown parameters (e.g. bias, stiffness). The KF tracks and updates the parameter estimate online. |
| Multi-step prediction | Run the time-update equations forward $j$ steps from $\hat{x}_{k\|k}$ without a measurement to get $\hat{x}_{k+j\|k}$ and its covariance. |
| Fault detection | Monitor the innovation $\tilde{y}_{k\|k-1}$; if it stops looking like white noise a fault is likely present. |
| Sensor fusion | Multiple sensors $\rightarrow$ multiple rows in $H_k$; the KF optimally combines all of them. |

### Challenges and limitations

- **Noise must be white.** The KF is optimal only when $w_k$ and $v_k$ are white (independent in time). *Coloured noise* (e.g. low-frequency drift) must be handled by augmenting the state with a noise model — for example adding a first-order Markov process driven by white noise.
- **Need to know $Q$ and $R$.** The noise covariances $Q_k$ and $R_k$ must be specified by the designer. If they can be measured or estimated from data, great; if not, they become tuning parameters. Wrong values make the filter suboptimal — or diverge.
- **Linear systems only.** For nonlinear systems the EKF (linearise around current estimate) or UKF (sigma points) must be used. See Lecture 5 for the IMU application.

**Summary — the Kalman filter in one paragraph**
The Kalman filter is the optimal (MMSE) recursive estimator for linear Gaussian systems. It alternates between a **prediction step** (propagate state and covariance forward using the dynamics model) and a **measurement update step** (correct using the innovation, weighted by the Kalman gain). The gain is computed from the covariance loop, which runs independently of the data and can be pre-solved for time-

invariant systems via the DARE. The filter is unbiased, its error is orthogonal to the measurements, and its innovations are white — any violation of these properties signals a model or sensor fault.